



AI Engineering Bootcamp · Session 1

Reliable Reasoning Components

Build your first production AI API: validated structured output, packaged in a container, deployed — by the end of today.

Dr Aki Wijesundara

Prompt · Context · Harness — all three today

Session 1 · Reliable Reasoning Components

01 / 34

Dr Aki Wijesundara



PhD and Postdoc in Machine Learning, King's College London

Part of Google's AI Accelerator

AI founder — SnapDrum and TAI Labs

Senior AI Advisor to the United Nations

Previous affiliations: MIT, Google



STANFORD



GOOGLE



SOCIAL CAPITAL



OXFORD



Session 1 · Reliable Reasoning Components

02 / 34

By the end of tonight, you can...

- 01 Explain the three layers — prompt, context, harness — and place a failure in one
- 02 Ship a FastAPI `/ask` endpoint that returns a validated Pydantic object
- 03 Stream tokens through FastAPI with `StreamingResponse`

- 04 Add timeouts, retries and a safe fallback so nothing returns a 500
 - 05 Containerize with Docker and deploy to a live URL
-

One capstone, six pillars



Session 1	Session 2	Session 3	Session 4	Session 5
Foundations	RAG	Agents	Evals + Memory	Production
Reliable API, deployed	Retrieval on top	Tools and memory		Demo Day

TRACE, long-running
state

Each session adds a layer to the same project — not five throwaway demos. By Demo Day: deployed, evaluated, agentic.

This session



Prompting lab — weak vs strong prompts, ready to copy

tailabs.ai/ai-eng-syllabus/prompting-lab



Cohort Slack — invite lands in your inbox tonight

Questions between sessions



Push every session's work to GitHub

Ongoing



Homework: containerize, deploy, 3 endpoints

Before Session 2

AI engineering is reliability around models, not model training

You build on frontier models you did not train: ground them in your data, give them tools, measure whether they work, and ship something that holds up with real users.

01 Foundations

What AI engineering is, the tools we use, and the three layers every LLM product has.

Goal 1

Session 1 · Reliable Reasoning Components

07 / 34

What AI engineering is — and is not

✗ Is not

Training models or prompt tinkering

You do not fine-tune GPT from scratch here

Clever prompts alone are not enough



✓ Is

Reliability around models

Context, schemas, retries, retrieval, agents

TRACE evals, memory, cost and latency

That stack is what makes a demo survive production

You compose APIs, data, tools and evals around models someone else trained

Cursor and Claude Code write most of the code. The scarce skill is **systems thinking, context engineering and judgment**: directing the agent, reading what it produced, deciding what to keep.

Session 1 · Reliable Reasoning Components

08 / 34

Everything we use this week, and why



Build & serve

FastAPI for the API, Pydantic for validated schemas, Cursor to build and test it, GitHub for every week's work.



AI & data

OpenAI API — gpt-5.5 for quality, gpt-5.4-mini for speed. Structured Outputs

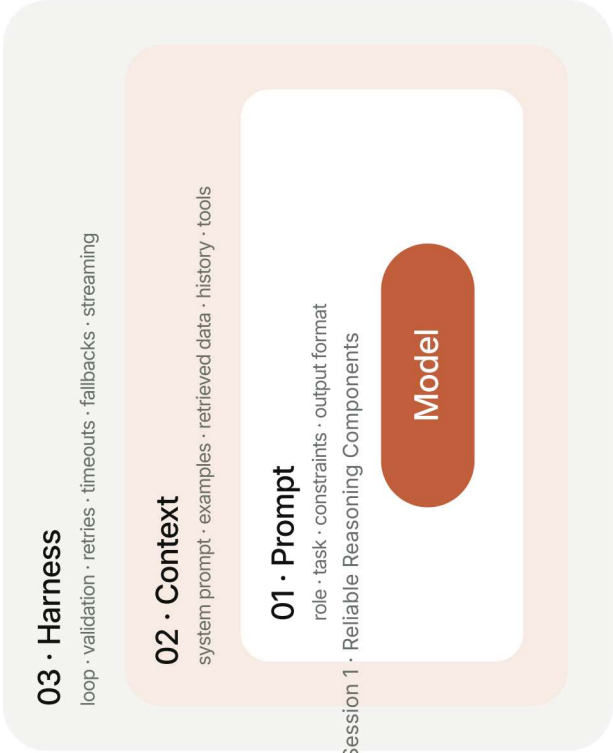


Infra & observability

Docker packages the app so it runs the same everywhere. Render, Railway or Fly: push a container, get a URL.

enforce the schema; streaming sends tokens as generated.

Three layers of AI engineering



Building with LLMs is three jobs, not one. The rest of the course goes deep on each.

Prompt: what you say to the model. Precise, not clever.

Context: what the model can see, and what you leave out.

10 / 34

Harness: what surrounds the model and makes it dependable.

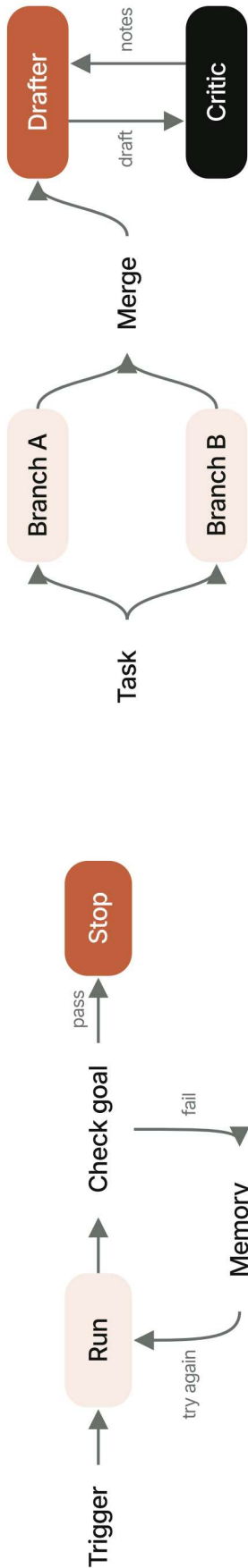
A great prompt inside a bad harness still fails in production.
Today we build all three.

Two newer shifts live **inside the harness**

Loop and graph engineering are not new layers. They are ways of working a harness you already have.

04 · Loop engineering

05 · Graph engineering



Session 1 · Reliable Reasoning Components

11 / 34

Run, check against a goal, remember, try again, stop. It repeats work without you.

Parallel branches, merge points, and a drafter kept separate from its critic.

Loop optimises repeated work; graph optimises coordination.

The car analogy

The model is the engine. Everything since 2022 is what people bolted on to turn an engine into a car.



Engine

The model.
Powerful, no
attachments.



Wheels + frame

Context
engineering: your



Dashboard + seatbelts

Harness
engineering: files,



Self-driving

Loop engineering:
take the human out
of the loop; it runs,



Road system

Graph engineering:
lanes and traffic
lights so many

Prompt engineering is learning to talk to it.	data, tools and memory make it move and do real work.	sandbox, logging, guardrails. Not just functional — safe.	checks and stops on its own.	agents work without colliding.
---	---	---	------------------------------	--------------------------------

Tonight we build the engine into a car: **prompt, context and harness**. Loops and graphs come once the car exists.

02

Context engineering

The model only knows what is in its context window right now.

Goal 1

Session 1 · Reliable Reasoning Components

13 / 34

Most “the model is dumb” problems are **context problems**

System prompt: the standing brief, including what to refuse

Examples: show what good output looks like

Retrieved documents: pulled in at query time

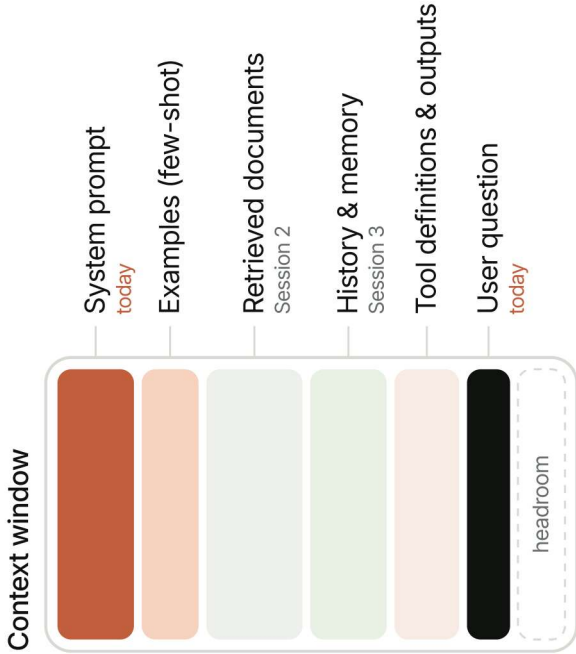
History & memory: too much is noise, too little loses track

14 / 34

Tools: what it can call, and the results

Today you control the system message, the question and optional context. RAG and agent memory are context engineering at scale.

Note

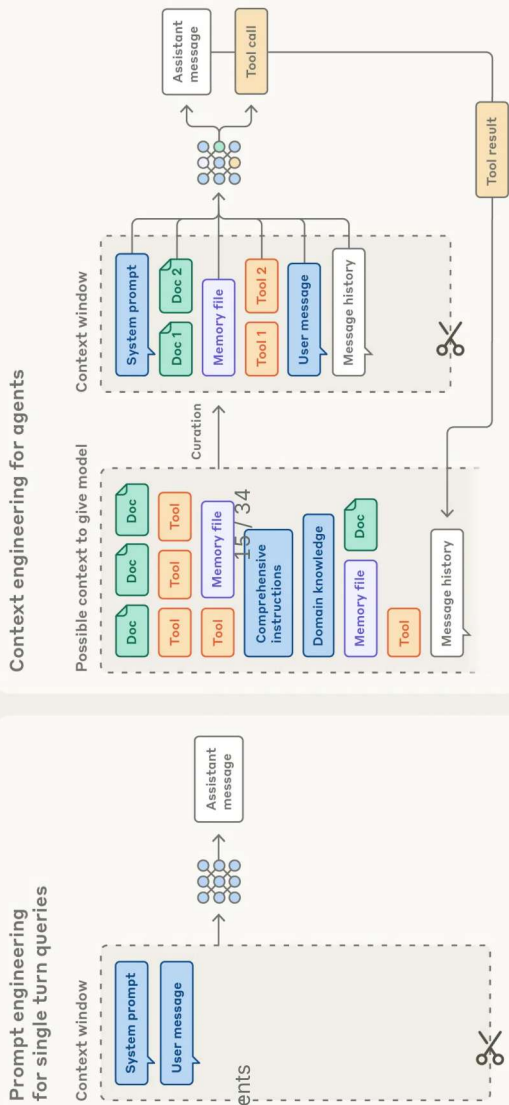


Fixed size: everything you add pushes something else out.



Prompt engineering vs context engineering

Prompt engineering vs. context engineering



Session 1 · Reliable Reasoning Components

03

OpenAI Playground

Same model, different brief. Sketch the API before you build it.

Goal 1 Goal 2

Session 1 · Reliable Reasoning Components

16 / 34



What we explore live

- 01 Specificity and role framing — weak vs strong: role, length, audience, tone, concrete facts
- 02 Few-shot prompting — examples pin labels and format tighter than instructions
- 03 Structured output — flip Text format to `json_schema` ; enforced at the API
- 04 Chain of thought + reasoning effort — auditable steps; try medium → high

05 Checkable constraints — “not too long” fails; “exactly three bullets under 25 words” you can verify

Lab

Prompts ready at tailabs.ai/ai-eng-syllabus/prompting-lab. Run weak first, then strong. The schema you sketch here is the Pydantic model you write in Part 3.

04

Build the API

Five files. Cursor writes the boilerplate; you understand every line.

Goal 2 Goal 3

Session 1 · Reliable Reasoning Components

18 / 34



Project structure: five files

File tree

```
research-assistant/  
├─ main.py           # FastAPI app  
├─ requirements.txt  # Dependencies  
├─ Dockerfile        # Container build  
├─ .env              # API keys (never commit)  
└─ README.md  
  
# requirements.txt  
fastapi  
openai  
pydantic
```

GET /ask response shape

```
{  
  "answer": "Prompt engineering is...",  
  "sources": ["source1", "source2"],  
  "confidence": 0.92,  
  "model": "gpt-5.4-mini",  
  "tokens_used": 150  
}
```

```
python-dotenv  
uvicorn
```

Production **.env** is in **.gitignore** — never commit your API key. Load it with **python-dotenv** .

Live coding with Cursor

```
main.py — skeleton

from fastapi import FastAPI
from openai import OpenAI
import os
app = FastAPI()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

@app.post("/ask")
async def ask_question(question: str):
    messages = [
        {"role": "system",
         "content": "You are a careful research assistant."},
```

Cursor prompts

- "Create a FastAPI app with a POST endpoint called /ask"
- "Add OpenAI integration using gpt-5.4-mini"
- "Add error handling and response validation"

```
    {"role": "user", "content": question},  
  ]  
  pass # completed together with Cursor
```

Production

Let Cursor write the boilerplate. Your job is to understand every line it writes — not to type it yourself.

Session 1 · Reliable Reasoning Components

20 / 34



Why stream: the first word lands early

The model takes the same time to finish either way. Streaming changes how long the user waits to see anything.

Without streaming



With streaming

Session 1 · Reliable Reasoning Components

Streaming: first words in under a second

Stream from the model

```
stream = client.chat.completions.create(  
    model="gpt-5.4-mini",  
    messages=[{"role": "user",  
                "content": question}],  
    stream=True,  
)  
for chunk in stream:  
    delta = chunk.choices[0].delta.content  
    if delta:  
        print(delta, end="", flush=True)
```

Stream through FastAPI

```
from fastapi.responses import StreamingResponse  
@app.post("/ask/stream")  
async def ask_stream(question: str):  
    def token_generator():  
        stream = client.chat.completions.create(  
            model="gpt-5.4-mini",  
            messages=[{"role": "user", "content": question}],  
            stream=True,  
        )  
        for chunk in stream:  
            delta = chunk.choices[0].delta.content
```

```
        if delta:
            yield delta
        return StreamingResponse(token_generator(), media_type="text/plain")
```

Rule

Stream for chat-style answers. Return validated JSON for machine-readable endpoints. We do both.

05

The harness

We have the prompt and the context. Now the code that makes the call dependable.

Goal 2

Goal 4

Session 1 · Reliable Reasoning Components

23 / 34

A raw model call is fragile

✗ Raw call

Fails in three ways

- Returns malformed or off-spec output
- Times out or gets rate-limited
- Occasionally just fails



✓ The harness we build

Fails cleanly, never silently

- Structured output — a guaranteed shape, not a string to parse
- Timeouts and retries — slow or failed calls fail cleanly

Graceful degradation — a sensible response, never a
500

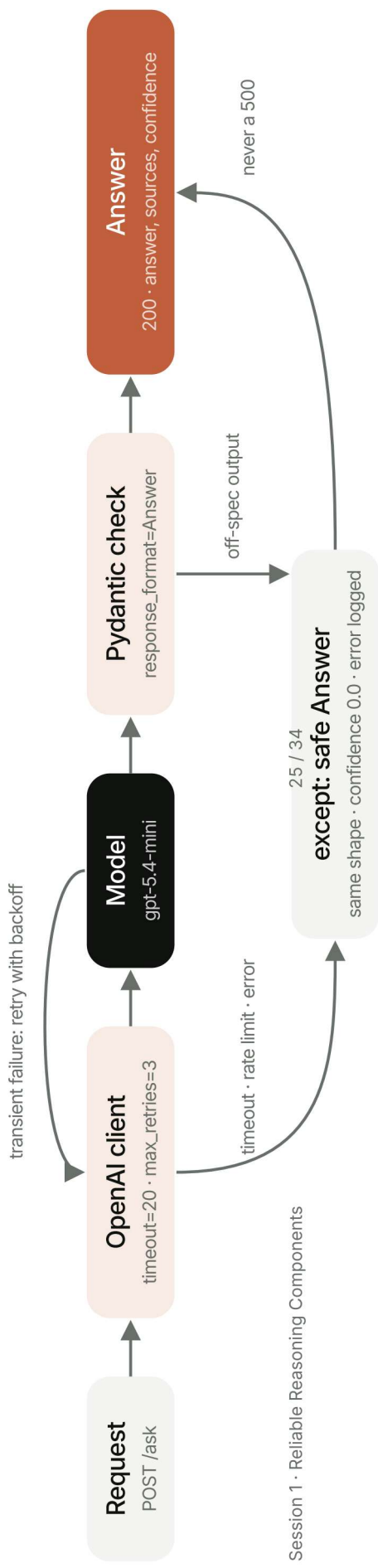
The model is the engine. The harness is the car. You ship the car. This is *why* the week is called Reliable Reasoning Components.

Session 1 · Reliable Reasoning Components

24 / 34



The harness around every call



Session 1 · Reliable Reasoning Components

Every path ends in the same Answer shape. The next two slides are the code for each box.

Structured outputs with Pydantic

Define the shape

```
from pydantic import BaseModel

class Answer(BaseModel):
    answer: str
    sources: list[str]
    confidence: float # 0.0 to 1.0
```

Ask the model to fill it

```
response = client.chat.completions.parse(
    model="gpt-5.4-mini",
    messages=[
        {"role": "system",
         "content": "You are a careful research assistant."},
        {"role": "user", "content": question},
    ],
    response_format=Answer,
)
result = response.choices[0].message.parsed
```

```
# result.answer, result.sources, result.confidence  
# parsed and validated — every field guaranteed
```

Production Putting “respond in JSON” in the prompt is a **hope**. Schema enforcement is a **guarantee**: no regex, no **json.loads**, no **KeyError**.



Retries, timeouts and graceful degradation

Never return a 500 — return a safe Answer

```

@app.post("/ask")
async def ask_question(question: str):
    try:
        response = client.chat.completions.parse(
            model="gpt-5.4-mini",
            messages=[{"role": "user", "content": question}],
            response_format=Answer,
        )
        return response.choices[0].message.parsed
    except Exception:
        # Log the real error. Still return 200 + a valid Answer.

```

Configure the client

```

client = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY"),
    timeout=20.0, # give up after 20s
    max_retries=3, # retry with backoff
)

# Harness checklist
# [x] validated output shape (Pydantic)
# [x] timeout set on the client
# [x] retries on transient failures

```

```
return Answer(answer="Sorry, I couldn't answer that right now.",           # [x] safe fallback on the bad path
               sources=[], confidence=0.0)                                # [x] errors logged, not swallowed
```

Upgrade

Optional: fall back to **gpt-5.5** if **gpt-5.4-mini** fails — a secondary client and catch specific exceptions.

Session 1 · Reliable Reasoning Components

27 / 34

06

Ship it

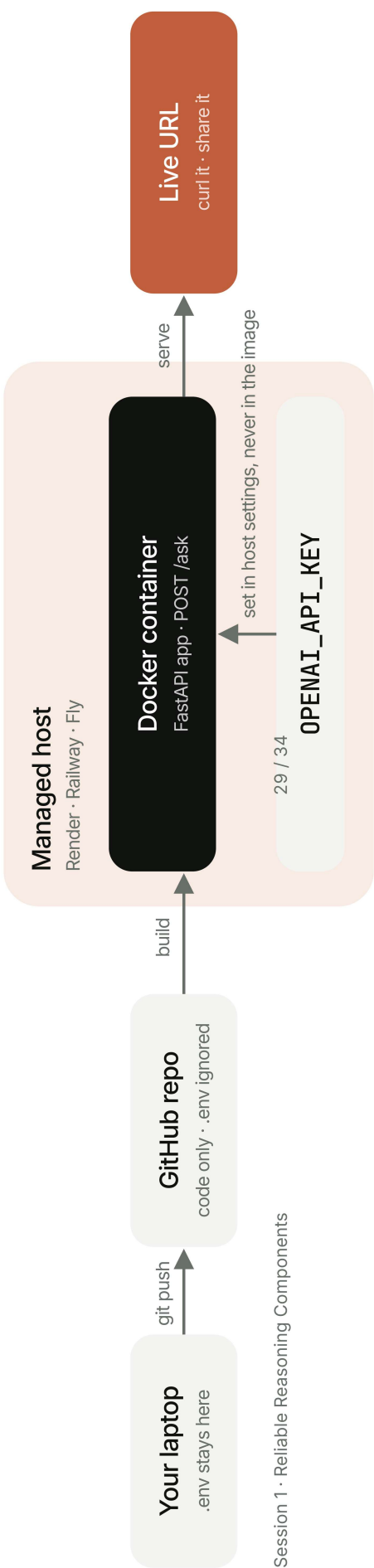
One container, one managed host, one live URL.

Goal 5

Session 1 · Reliable Reasoning Components

28 / 34

What you ship tonight



Session 1 · Reliable Reasoning Components

Containerize with Docker

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["fastapi", "run", "main.py",
    "--host", "0.0.0.0", "--port", "8000"]
```

Build and run locally

```
# build the image
docker build -t research-assistant .

# run it with your env vars
docker run -p 8000:8000 \
    --env-file .env \
    research-assistant
```

Note

A container packages your app and its dependencies so it runs the same everywhere — the standard unit of deployment for Render, Railway, Fly and Cloud Run.

Then deploy

Push to GitHub

Render, Railway and Fly connect to your repo directly



Connect repo, set env vars

`OPENAI_API_KEY` goes in the host's settings — never in the image



Get a live URL

Share it. Test it with Cursor's agent. Curl it. It's real.

Not today

At scale, orchestrators like Kubernetes, Cloud Run or ECS run many containers. For your first deploy, one container on a managed host is plenty.

Session 1 · Reliable Reasoning Components

31 / 34

What you can do now

- 01 Explain the three layers — prompt, context, harness — and place a failure in one
- 02 Ship a FastAPI `/ask` endpoint that returns a validated Pydantic object
- 03 Stream tokens through FastAPI with `StreamingResponse`

- ✓ Prompt = what you say · Context = what it sees · Harness = what surrounds it
- ✓ `response_format=Answer` is a contract, not a hope — every field guaranteed
- ✓ Stream for chat answers; return validated JSON for machine endpoints

- 04** Add timeouts, retries and a safe fallback so nothing returns a 500  **Timeout + retries on the client, same Answer shape on the bad path**
- 05** Containerize with Docker and deploy to a live URL  **One Dockerfile, one managed host, one URL you can curl tonight**

Homework: ship the full thing

RAG pipelines next week — you need a working, deployed API before we add retrieval on top.

- | | | | |
|-----------|---|-----------|---|
| 01 | Containerize and run locally with <code>--env-file .env</code> | 02 | Deploy to Render, Railway or Fly.io — a live URL you can share |
| 03 | Add <code>/summarize</code> and <code>/analyze-sentiment</code> , both returning validated Pydantic objects | 04 | Add <code>/ask/stream</code> with <code>StreamingResponse</code> — confirm tokens arrive in real time |

05 Every endpoint: timeout, retry, graceful fallback — no endpoint returns a 500

Effort 3–5 hrs Due Before Session 2 Stretch Read up on RAG · model fallback to gpt-5.5

Session 1 · Reliable Reasoning Components

33 / 34



Questions?

Recording + prompting lab: cohort Slack tonight

Next · Session 2

RAG: retrieval on top of your API

A vector database and a retrieval layer over the endpoints you deploy this week.

